J Optim Theory Appl (2016) 170:591–615
DOI 10.1007/s10957-016-0890-5

Pooling Problems with Polynomial-Time Algorithms

Dag Haugland[1] · **Eligius M. T. Hendrix**[2,3]

Received: 19 June 2015 / Accepted: 1 February 2016 / Published online: 19 February 2016
© Springer Science+Business Media New York 2016

Abstract The computational challenge offered by many traditional network flow models is modest, and large-scale instances can be solved fast. When the composition of the flow is part of the model, the required computation time may increase substantially. This is in particular true for the pooling problem, where the relative content of certain flow components is restricted. Flow entering the network at the source nodes has a given composition, whereas the composition in other nodes is determined by the composition of entering flows. At the network terminals, the flow composition is subject to restrictions referred to as quality constraints. The pooling problem is known to be strongly NP-hard, even if the network has only one pool, but is solvable in polynomial time if also the number of terminals or the number of quality parameters (flow components) is bounded. The problem is also NP-hard if there are only two sources and terminals and only one quality parameter. Two related questions have been left open in the literature so far. For the single-pool version, it has not been known whether the problem is solvable in polynomial time if the number of sources is bounded. For the version with a single quality parameter and two sources and terminals, the question whether a pseudo-polynomial algorithm exists has been open. This paper gives positive answers to both questions.

Communicated by Michael Patriksson.

✉ Dag Haugland
dag@ii.uib.no

Eligius M. T. Hendrix
eligius.hendrix@wur.nl

1 Department of Informatics, University of Bergen, Bergen, Norway

2 Department of Computer Architecture, Universidad de Málaga, Málaga, Spain

3 Operations Research and Logistics Group, Wageningen University, Wageningen, Netherlands

Keywords Pooling problem · Complexity · Global optimization · Polynomial-time algorithm

Mathematics Subject Classification 90C26 · 90C35 · 90C39 · 68Q25

List of Symbols

$\mathbb{R}^n$	The n-dimensional space of real vectors
$\mathbb{Q}^n$	The n-dimensional space of rational vectors
$\mathcal{D} = (\mathcal{N}, \mathcal{A})$	Directed graph with node set $\mathcal{N}$ and arc set $\mathcal{A}$
S	Set of source nodes
I	Set of pool nodes
T	Set of terminal nodes
K	Set of quality parameters
S_i	Set of neighbor sources of pool i
T_i	Set of neighbor terminals of pool i
I_j	Set of neighbor pools of node j
$\mathcal{A}_S$	Set of source-incident arcs
$\mathcal{A}_T$	Set of terminal-incident arcs
$\boldsymbol{Q} = \left(q_s^k\right)_{s\in S, k\in K}$	Matrix of source qualities
$\boldsymbol{U} = \left(u_t^k\right)_{t\in T, k\in K}$	Matrix of terminal quality bounds
$\boldsymbol{b}$	Vector of node capacities
$\boldsymbol{c}$	Vector of arc costs
s_0	A source for which $q_{s_0}^k = 0$ for all $k \in K$
S_0	$S\backslash\{s_0\}$
$\boldsymbol{f}$	Flow vector
$\boldsymbol{W} = \left(w_j^k\right)_{j\in\mathcal{N}, k\in K}$	Matrix of flow qualities
$\mathcal{F}(\mathcal{D}, \boldsymbol{b})$	Set of all flow vectors satisfying conservation of flow and node capacities
$\mathcal{T}^+ : \mathcal{F}(\mathcal{D}, \boldsymbol{b}) \Rightarrow 2^T$	Maps flow vector $\boldsymbol{f}$ to the set of terminals with all quality constraints satisfied by $\boldsymbol{f}$
$\mathcal{T}(\mathcal{D}, \boldsymbol{b})$	Image of $\mathcal{T}^+$
x_s	Proportion of the flow through a pool originating from source s
$z\left(T^+\right)$	Minimum cost when only terminals in T^+ can receive positive flow
P	Polytope in $\mathbb{R}^n$
V	Finite set in $\mathbb{R}^n$
$V(P)$	Vertex set of polytope P
$P(V)$	Convex hull of V
$\boldsymbol{A} = \left(a_{ij}\right)_{i\in M, j\in N}$	Rational matrix
$\boldsymbol{d} = (d_i)_{i\in M}$	Rational vector
$\left(\bar{\mathbf{A}}, \bar{\mathbf{d}}\right)$	The system $(\mathbf{A}, \mathbf{d})$ extended by rows with coefficients of the inequalities defining the unit simplex
$(M^<, M^>)$	Valid cover

Δ^n	Unit n-simplex
$\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$	Polyhedral complex induced by $(\boldsymbol{A}, \boldsymbol{d})$
$\mathcal{P}^r(\boldsymbol{A}, \boldsymbol{d})$	Set of r-dimensional polyhedra in $\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$
$\mathcal{V}(\boldsymbol{A}, \boldsymbol{d})$	Vertex sets of polyhedra in $\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$
$\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})$	Vertex sets of polyhedra in $\mathcal{P}^r(\boldsymbol{A}, \boldsymbol{d})$
$\bar{\mathcal{D}} = (\bar{\mathcal{N}}, \bar{\mathcal{A}})$	Directed graph in an atomic instance
$(\bar{S}, \bar{I}, \bar{T})$	(Sources, pools, terminals) in an atomic instance
$(J_2, \ldots, J_6)$	Partition of I
$\boldsymbol{B} = (B_2, \ldots, B_6)$	Total capacities of pools in $J_2, \ldots, J_6$
$\zeta(\mathcal{D}, \boldsymbol{b})$	Minimum cost of flow in network $\mathcal{D}$ with node capacities $\boldsymbol{b}$ (dependency on other parameters suppressed)
$(e_2, \ldots, e_6)$	Unit vector representing the choice of one of the pool sets $J_2, \ldots, J_6$
$E(\boldsymbol{B})$	Set of feasible values of $(e_2, \ldots, e_6)$ when $J_2, \ldots, J_6$ have capacities $B_2, \ldots, B_6$
$z_n(\boldsymbol{B})$	Minimum cost when $J_2, \ldots, J_6$ have capacities $B_2, \ldots, B_6$ and $n-1$ pools are assigned to a set in $(J_2, \ldots, J_6)$
$\nu(\mathcal{D}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{q}, \boldsymbol{u})$	Size of a problem instance
p	Polynomial function of the instance size

1 Introduction

Traditional network flow models, like the minimum cost network flow problem, rely on considerations of the flow as a homogeneous commodity. Variations in composition is, however, relevant in many industrial and logistics applications of the models, including petroleum refining and pipeline transportation of natural gas. Crude oils supplied to a refinery have contamination levels depending on their sources of origin. Major components of natural gas are methane, ethane, butane and propane, but the proportions in which they occur are not equal for all gas wells. For environmental or technical reasons, requirements to the final composition are often imposed at the terminal nodes of the flow network. Whenever constraints of this kind apply, it is therefore essential for network flow models to recognize not only the total product flow, but also how the flow composition evolves from network sources to terminals. In particular, the models must reflect updates of the composition at nodes where differently composed flows are pooled. This provides a computationally challenging problem referred to as the *pooling problem*.

The pooling problem exhibits strong resemblance with well-studied logistics models like the minimum cost flow problem and the transportation problem. Another classical-related problem is Stigler's diet problem [1,2], asking for the minimum cost diet requiring a certain amount of specified nutrients. The problem is easy to extend to a multi-diet version where groups of people have each their own requirements. Nutritional contents differ between sources, that is, foods of which diets can be composed, and the optimal mix must be found for each terminal (group of people). Conceptually, the multi-diet version coincides with the pooling problem in bipartite networks. Thanks to this simple network structure, with exclusively direct links between sources

and terminals, diet problems are nothing but linear programs. However, when extending the problem to more general network structures, this is no longer the case. The absence of linear formulations has immediate computational consequences. While minimum cost flow instances where the network size reaches a six digit number of arcs and nodes can be solved within hours, or even minutes, exact solution of pooling problem instances with hundreds of nodes appears to be unrealistic.

The pooling problem has been recognized for decades as a challenge where linear programming approaches fail. Bounds on the flow composition frequently induce bilinear constraints in otherwise linear models and require corresponding solution methods. An approach based on iterated solution of linearized model versions has gained popularity, especially in the oil refining industry, and has been analyzed experimentally by Haverly [3]. A major conclusion is that methods of this kind may produce suboptimal solutions. Such observations helped establishing a consensus, already in early days of research in the pooling problem, that global optimality is achievable only in small instances.

Owing to the anticipated intractability of the problem, early research was mainly directed toward heuristic methods [3–5], where iterative linearization is the core idea. Floudas and Visweswaran [6] reported the first exact solution algorithm for the pooling problem. Later, algorithms based on branch-and-bound [7–11], Lagrangian relaxation [12–14], and integer programming [15] have been studied. The industrial relevance of the problem, notably in petroleum refining [4,5], the food industry [16], and in waste water processing [17,18], has been acknowledged by many authors. The pooling problem survey by Misener and Floudas [19] has a comprehensive list of references to work in this area.

Standard pooling problems are defined as networks with three node layers, referred to as sources, pools, and terminals (sinks), respectively. Each arc links either a source to a pool or a pool to a terminal. Quality constraints in terms of bounds on the composition are imposed at the terminals. Each bound relates to the relative content of a flow component, referred to as the quality parameter. Rigorous studies of the computational complexity of the pooling problem have not been conducted until recently. After NP-hardness was formally proved [20], Dey and Gupte [15] showed that the problem is as hard to approximate as the maximum clique problem.

The hardness theorem in Alfaki and Haugland [20] states that even the class of instances with a unique pool is strongly NP-hard. The computational complexity is, however, favorable if there are additional limitations on node cardinalities, as polynomial-time algorithms have been developed for the case of a unique pool and an upper bound on either the number of terminals [21] or the number of quality parameters [20]. Whether polynomial running time algorithms for the one-pool instance class exist when the number of sources is bounded, has, however, been an open question [21].

The pooling problem remains NP-hard for networks with only two sources and two terminals, and only one flow component subject to constraints [21]. Whether such instances can be solved in polynomial time when the input data take values within given bounds has been an open question to date. In the present work, we answer both of the questions above in the affirmative. First, we present a new algorithm for the instance class with a unique pool and prove that it runs in polynomial time when the

source cardinality is fixed. Second, we give a dynamic programming algorithm for a rich class of instances with a unique quality parameter and only two sources and terminals. This algorithm is proved to have pseudo-polynomial running time, which implies that its running time is polynomial for any instance class in which the values of all numerical input data are bounded. Contributions to the pooling problem literature from the current text are thus answers to two questions left open in existing literature, and, by virtue of this, an enhanced understanding of the boundaries between tractable and intractable versions of the problem.

The remainder of the text is organized as follows: Mathematical notation and formulation of the pooling problem are provided in Sect. 2. Section 3 reviews the complexity of the instance class with a single pool. In Sect. 4, we consider a set of hyperplanes intersecting the unit simplex and introduce the problem of enumerating all polytopes in the resulting partition of the simplex. Results from an analysis of this problem become useful when analyzing the complexity of single-pool instances with bounded source cardinality. For this instance class, we prove solvability in polynomial time. Instances with a unique quality parameter, only two sources and terminals, and a specific cost structure are considered in Sect. 5. In terms of a recursion formula, we show how any such instance can be reduced to an instance of constant size. The reduction leads to a dynamic programming algorithm, which is proved to run in pseudo-polynomial time. Main conclusions and future outlooks are discussed in Sect. 6.

2 Mathematical Notation and Problem Formulation

Let $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ be a directed graph, where the node set $\mathcal{N} = S \cup I \cup T$ consists of the disjoint subsets S of *sources*, I of *pools*, and T of *terminals*. The arc set $\mathcal{A} \subseteq (S \times I) \cup (I \times T)$ links sources with pools and pools with terminals. Let K be a finite set and refer to its elements as *quality parameters*. In the sequel, components of vectors and matrices defined over the sets $\mathcal{N}$ and K are identified by a subscript if they correspond to $\mathcal{N}$ and by a superscript if they correspond to K. Complexity analyses in later sections require that network parameters are defined as rational rather than real numbers. Define the vectors of unit arc costs $\boldsymbol{c} \in \mathbb{Q}^{|\mathcal{A}|}$ and node capacities $\boldsymbol{b} \in \mathbb{Q}_+^{|\mathcal{N}|}$. For integers j and n, let $\boldsymbol{j}^n$ denote the vector with all its n entries equal to j, and let $\boldsymbol{I}^n$ denote the $(n \times n)$-identity matrix.

At all sources $s \in S$, flow with a given quality may enter $\mathcal{D}$, and flow leaving the network at a terminal $t \in T$ must satisfy given quality bounds. Define the matrices of *source quality* $\boldsymbol{Q} \in \mathbb{Q}^{|K|\times|S|}$ and *terminal quality bounds* $\boldsymbol{U} \in \mathbb{Q}^{|K|\times|T|}$. For $s \in S$ and $t \in T$, let $\boldsymbol{q}_s = \left(q_s^k\right)_{k\in K}$ and $\boldsymbol{u}_t = \left(u_t^k\right)_{k\in K}$ be the columns of $\boldsymbol{Q}$ and $\boldsymbol{U}$, respectively. That is, q_s^k is the quality in terms of parameter k of the flow entering $\mathcal{D}$ at s, and u_t^k is the upper bound on the quality (smaller numerical values indicate better quality) of the flow leaving $\mathcal{D}$ at t.

For each pool $i \in I$, the set of neighbor sources and the set of neighbor terminals are denoted by $S_i = \{s \in S\colon (s,i) \in \mathcal{A}\}$ and $T_i = \{t \in T\colon (i,t) \in \mathcal{A}\}$, respectively. For each source $s \in S$, the set of neighbor pools is denoted by $I_s = \{i \in I\colon (s,i) \in \mathcal{A}\}$, while $I_t = \{i \in I\colon (i,t) \in \mathcal{A}\}$ denotes the set of neighbor pools of terminal $t \in T$. The

arc set $\mathcal{A}$ has a partition $(\mathcal{A}_S, \mathcal{A}_T)$, where $\mathcal{A}_S = \mathcal{A} \cap (S \times I)$ and $\mathcal{A}_T = \mathcal{A} \cap (I \times T)$ denote the sets of source-incident and terminal-incident arcs, respectively.

A vector $\boldsymbol{f} \in \mathbb{R}_+^{|\mathcal{A}|}$ is called a *flow* in $\mathcal{D}$ iff it satisfies the capacity and flow conservation constraints:

$$\sum_{i \in I_s} f_{si} \leq b_s \qquad s \in S, \tag{1}$$

$$\sum_{s \in S_i} f_{si} \leq b_i \qquad i \in I, \tag{2}$$

$$\sum_{i \in I_t} f_{it} \leq b_t \qquad t \in T, \tag{3}$$

$$\sum_{s \in S_i} f_{si} - \sum_{t \in T_i} f_{it} = 0 \qquad i \in I. \tag{4}$$

We refer to the set $\mathcal{F}(\mathcal{D}, \boldsymbol{b}) = \left\{ \boldsymbol{f} \in \mathbb{R}_+^{|\mathcal{A}|} : (1)\text{–}(4) \text{ are satisfied} \right\}$ as the *flow polytope* of $\mathcal{D}$. For each $\boldsymbol{f} \in \mathcal{F}(\mathcal{D}, \boldsymbol{b})$, define $\boldsymbol{W} = \left(\boldsymbol{w}_j\right)_{j \in \mathcal{N}} \in \mathbb{R}^{|K| \times |\mathcal{N}|}$ to be a *quality matrix* of $\boldsymbol{f}$ iff $\boldsymbol{w}_s = \boldsymbol{q}_s$ $(s \in S)$,

$$\boldsymbol{w}_i \sum_{t \in T_i} f_{it} = \sum_{s \in S_i} \boldsymbol{q}_s f_{si}, \ i \in I, \text{ and}$$

$$\boldsymbol{w}_t \sum_{i \in I_t} f_{it} = \sum_{i \in I_t} \boldsymbol{w}_i f_{it}, \ t \in T.$$

That is, the quality $\boldsymbol{w}_i = \left(w_i^k\right)_{k \in K}$ of the flow blended at a pool i receiving positive flow, is a convex combination of the source qualities $\boldsymbol{q}_s$ $(s \in S)$, with convexity weights $f_{si} / \sum_{t \in T_i} f_{it}$ $(s \in S_i)$. Analogously, for terminals $t \in T$ with positive inflow, the quality $\boldsymbol{w}_t$ is a convex combination of the pool qualities $\boldsymbol{w}_i$ $(i \in I_t)$. These conditions on $\boldsymbol{w}$ are referred to as the *linear blending* property, and imply directly

$$\sum_{s \in S} \boldsymbol{q}_s \sum_{i \in I_s} f_{si} = \sum_{t \in T} \boldsymbol{w}_t \sum_{i \in I_t} f_{it}. \tag{5}$$

At terminals $t \in T$ where $\sum_{i \in I_t} f_{it} > 0$, the upper quality bounds u_t^k apply, and linear blending yields $\boldsymbol{w}_t = \sum_{i \in I_t} \boldsymbol{w}_i f_{it} / \sum_{i \in I_t} f_{it} \leq \boldsymbol{u}_t$. Thus, $\boldsymbol{f} \in \mathcal{F}(\mathcal{D}, \boldsymbol{b})$ is said to be a *feasible* flow in $\mathcal{D}$ iff $\sum_{i \in I_t} \boldsymbol{w}_i f_{it} \leq \boldsymbol{u}_t \sum_{i \in I_t} f_{it}$ $(t \in T)$ for any quality matrix $\boldsymbol{W}$ of $\boldsymbol{f}$.

Problem 2.1 The POOLING PROBLEM: Find a feasible flow $\boldsymbol{f}$ in $\mathcal{D}$ minimizing the total costs $\boldsymbol{c} \cdot \boldsymbol{f}$. □

Problem 2.1 is traditionally stated in terms of a bilinear formulation in flow and pool quality variables [3], often referred to as the P-formulation:

$$\min_{\boldsymbol{f}, \boldsymbol{W}} \left(\sum_{(s,i) \in \mathcal{A}_S} c_{si} f_{si} + \sum_{(i,t) \in \mathcal{A}_T} c_{it} f_{it} \right), \tag{6}$$

$$\sum_{s\in S_i} q_s^k f_{si} - w_i^k \sum_{t\in T_i} f_{it} = 0 \qquad i \in I, k \in K, \tag{7}$$

$$\sum_{i\in I_t} w_i^k f_{it} - u_t^k \sum_{i\in I_t} f_{it} \le 0 \qquad t \in T, k \in K, \tag{8}$$

$$\boldsymbol{f} \in \mathcal{F}(\mathcal{D}, \boldsymbol{b}). \tag{9}$$

3 Complexity of the Pooling Problem with a Single Pool

In this section, we assume that $\mathcal{A} = (S \times I) \cup (I \times T)$ and $|I| = 1$, which means that all sources and all terminals are connected to a unique pool in the network. A pool notation thus becomes superfluous, and we let f_s and f_t denote, respectively, the flow from source $s \in S$ to the pool, and flow from the pool to terminal $t \in T$. An analogous simplified notation is adopted for the arc costs.

3.1 Problem Formulation with a Set-Valued Variable

Because a terminal t receives flow exclusively from the unique pool, either $f_t = 0$, or the flow entering the pool must make a blend that satisfies the bounds u_t^k, $k \in K$. Hence, we have the following disjunctive constraint for all $t \in T$:

$$\begin{array}{ll} f_t = 0 & \text{or} \\ \sum_{s\in S} q_s^k f_s - u_t^k \sum_{s\in S} f_s \le 0 & k \in K. \end{array} \tag{10}$$

Considering the set $T^+ \subseteq T$ of terminals receiving positive flow as a variable, the problem is formulated as:

$$\min_{T^+, \boldsymbol{f}} \left(\sum_{s\in S} c_s f_s + \sum_{t\in T} c_t f_t \right), \tag{11}$$

$$\boldsymbol{f} \in \mathcal{F}(\mathcal{D}, \boldsymbol{b}), \tag{12}$$

$$\sum_{s\in S} q_s^k f_s - u_t^k \sum_{s\in S} f_s \le 0 \; k \in K, t \in T^+, \tag{13}$$

$$f_t = 0 \qquad t \in T \setminus T^+, \tag{14}$$

$$T^+ \subseteq T. \tag{15}$$

It has been shown [20] that the INDEPENDENT VERTEX SET problem is polynomially reducible to problem (11)–(15), which hence is (strongly) NP-hard. By exploiting the same reduction, it was recently shown [15] that the problem has no polynomial-time approximation algorithm within $|T|^{1-\epsilon}$, for any $\epsilon > 0$, unless NP-complete problems have randomized polynomial algorithms.

The computational difficulty of problem (11)–(15) is represented by the set-valued variable T^+, and once $T^+ \subseteq T$ is known, the remaining problem (11)–(14) is a linear program (LP). This observation inspires various solution approaches discussed in the next sections.

For a given $T^+ \subseteq T$, let $z(T^+)$ denote the minimum cost in the linear program (11)–(14).

3.2 Few Terminals

If the number $|T|$ of terminals is small, problem (11)–(15) can be solved by enumerating all $2^{|T|}$ possible $T^+ \subseteq T$. Taking the minimum of the $2^{|T|}$ LP-solutions $z\left(T^+\right)$, yields the optimal solution. Consequently, the POOLING PROBLEM with a single pool and an upper bound on the number $|T|$ of terminals is solvable in polynomial time [21].

3.3 Few Quality Parameters

A small number $|K|$ of quality parameters suggests enumeration of all possible pool qualities rather than enumeration of terminal sets. Consider a quality parameter $k \in K$ and a permutation $\pi\colon \{1,\ldots,|T|\} \mapsto T$ such that $u^k_{\pi(1)} \leq u^k_{\pi(2)} \leq \cdots \leq u^k_{\pi(|T|)}$. That is, terminal $\pi(1)$ imposes the strictest and terminal $\pi(|T|)$ the least strict quality bound on parameter k. If $u^k_{\pi(j-1)} \sum_{s\in S} f_s < \sum_{s\in S} q^k_s f_s \leq u^k_{\pi(j)} \sum_{s\in S} f_s$, then the pool quality satisfies the k-bound at terminals $\pi(j),\ldots,\pi(|T|)$, whereas terminals $\pi(1),\ldots,\pi(j-1)$ must receive zero flow. Letting j run through all possible values $1,\ldots,|T|+1$, where $j = |T|+1$ indicates that none of the bounds are satisfied, yields $|T|+1$ possible pairs $\left(\sum_{s\in S} q^k_s f_s - u^k_{\pi(j)} \sum_{s\in S} f_s \leq 0,\ f_{\pi(1)} = \cdots = f_{\pi(j-1)} = 0\right)$ of constraints for this particular $k \in K$. Combining all $|K|$ quality parameters gives $(|T|+1)^{|K|}$ compact LPs to solve. It follows that the POOLING PROBLEM with a single pool and an upper bound on the number $|K|$ of quality parameters is also solvable in polynomial time [20].

3.4 Linearly Ordered Quality Bounds

Consider instances with total domination in the quality bounds $\boldsymbol{u}_t$. This means that the terminals can be ordered such that they appear successively less demanding in all quality parameters k and implies that the permutation π discussed in Sect. 3.3 is independent of k. For the class of such single-pool instances, there exists an algorithm with polynomial running time. This is stated more precisely as follows:

Proposition 3.1 *If there exists a bijection* $\pi\colon \{1,\ldots,|T|\} \mapsto T$ *such that* $\boldsymbol{u}_{\pi(1)} \leq \cdots \leq \boldsymbol{u}_{\pi(|T|)}$, *then the* POOLING PROBLEM *with* $|I| = 1$ *is solved in terms of* $|T|$ *compact linear programs.*

Proof Let $T^+_j = \{\pi(j),\ldots,\pi(|T|)\}$ $(j = 1,\ldots,|T|)$ and $T^+_j = \emptyset$ for $j = |T|+1$. The proposition follows directly by observing that the optimal solution to (11)–(15) is $\min_{j=1,\ldots,|T|+1} z\left(T^+_j\right)$. □

Remark 3.1 Proposition 3.1 also motivates heuristic approaches aimed for instances where there exists no bijection satisfying the given criterion. Let π be defined such that some measure of the quality demand at terminal $\pi(j)$ is non-increasing with increasing value of j. Then it is reasonable to assume that for some $j = 1, \ldots, |T|+1$, the set T_j^+ as defined in the above proof will give a small cost $z(T_j^+)$. The heuristic method hence amounts to solving the $|T|$ associated linear programs. A natural measure of quality demand is $\sum_{k\in K} \frac{u_t^k}{\max_{s\in S} q_s^k - \min_{s\in S} q_s^k}$. It is also possible to apply the indicated heuristic for a set of different measures, e.g., by ordering the terminals by increasing value of u_t^k, and thus repeat the heuristic for all $k \in K$. □

3.5 Few Sources

Whether or not there exists an algorithm with polynomial running time for the POOLING PROBLEM with one pool ($|I| = 1$) and an upper bound on the number of sources $|S|$ has been an open question to date [21]. To prepare for an answer in the affirmative, define the map $\mathcal{T}^+ : \mathcal{F}(\mathcal{D}, \boldsymbol{b}) \Rightarrow 2^T$ from the flow polytope to the set of terminal subsets, such that

$$\mathcal{T}^+(\boldsymbol{f}) = \{t \in T : \text{ (10) is satisfied}\}.$$

This means that for a given flow $\boldsymbol{f}$, $\mathcal{T}^+(\boldsymbol{f}) \subseteq T$ consists of *all* terminals $t \in T$ such that $\boldsymbol{f}$ satisfies the quality constraints of t. A set $T^+ \subseteq T$ is thus in the image $\mathcal{T}(\mathcal{D}, \boldsymbol{b})$ of $\mathcal{T}^+$, if and only if there exists an $\boldsymbol{f} \in \mathcal{F}(\mathcal{D}, \boldsymbol{b})$ satisfying all quality constraints of all $t \in T^+$, while violating at least one quality constraint of each $t \in T \setminus T^+$. For instance, under the condition considered in Proposition 3.1, $\mathcal{T}(\mathcal{D}, \boldsymbol{b}) = \left\{T_j^+ : j = j_0, \ldots, j_1\right\}$ for some $1 \le j_0 \le j_1 \le |T| + 1$.

Proposition 3.2 $\min\left\{z\left(T^+\right) : T^+ \subseteq T\right\} = \min\left\{z\left(T^+\right) : T^+ \in \mathcal{T}(\mathcal{D}, \boldsymbol{b})\right\}$.

Proof Because $\boldsymbol{f} = \boldsymbol{0}^{|\mathcal{A}|}$ satisfies T, $T \in \mathcal{T}(\mathcal{D}, \boldsymbol{b})$, and the latter minimum exists. Let $T^+ \in 2^T \setminus \mathcal{T}(\mathcal{D}, \boldsymbol{b})$. Then, any optimal flow in (11)–(14) satisfies some $T^* \in \mathcal{T}(\mathcal{D}, \boldsymbol{b})$, where $T^* \supset T^+$. It follows that $z(T^*) \le z\left(T^+\right)$. □

The enumeration algorithm indicated in Sect. 3.2 computes $z(T^+)$ for *every* $T^+ \subseteq T$. According to Proposition 3.2, however, it suffices to compute $z(T^+)$ for all $T^+ \in \mathcal{T}(\mathcal{D}, \boldsymbol{b})$. This is exploited in the algorithm detailed and analyzed in the next section. Particularly, when $|S|$ is fixed, it is proved that $|\mathcal{T}(\mathcal{D}, \boldsymbol{b})|$ is polynomial in the instance size. In its turn, this leads to a proof of polynomial running time of the algorithm.

4 The Pooling Problem with One Pool and Fixed Number of Sources

4.1 Preliminaries

For a nonzero flow $\boldsymbol{f}$, let $x_s = \frac{f_s}{\sum_{t\in T} f_t}$ denote the proportion [14] of the total flow originating from source s. Without loss of generality [21, Proposition 2], we assume

that, for some $s_0 \in S$, $\boldsymbol{q}_{s_0} = \boldsymbol{0}^{|K|}$. Letting $S_0 = S \setminus \{s_0\}$, we thus have that the proportion vector $\boldsymbol{x}$ takes values on the unit simplex:

$$\sum_{s \in S_0} x_s \leq 1, \tag{16}$$

$$x_s \geq 0 \quad s \in S_0. \tag{17}$$

The slack in (16) equals the flow proportion corresponding to source s_0.

Dividing (10) by the total flow $\sum_{s \in S} f_s = \sum_{t \in T} f_t$ shows that if $T^+ \in \mathcal{T}(\mathcal{D}, \boldsymbol{b})$, then there exists an $\boldsymbol{x}$ on the unit simplex such that for all $t \in T^+$:

$$\sum_{s \in S_0} q_s^k x_s \leq u_t^k, k \in K. \tag{18}$$

The hyperplanes satisfying $\sum_{s \in S_0} q_s^k x_s = u_t^k$ for any $t \in T$ and $k \in K$ partition the unit simplex into a set of polytopes. Across the relative interior of any such polytope, the set T^+ of terminals t for which (18) is satisfied is invariant. If all polytopes are available, problem (11)–(15) is solved by the following operations on each of them: First, the corresponding set T^+ is identified. This is accomplished by choosing an arbitrary relative interior point $\boldsymbol{x}$ and identifying T^+ as $\{t \in T : (18) \text{ is satisfied}\}$. With T^+ fixed, the resulting LP (11)–(14) is solved next. The best solution hence obtained is also the solution to (11)–(15).

The outlined approach calls for an algorithm that takes a set of linear equations as input and outputs the set of polytopes in the corresponding partition of the unit simplex. For the case of the Pooling Problem with $|I| = 1$, such an algorithm is illustrated in the next section. The problem of identifying the partition of the simplex induced by intersecting hyperplanes is analyzed in general in Sect. 4.3. The analysis leads to an algorithm presented in Sect. 4.4, and we prove that its running time is polynomial in the number of hyperplanes when the variable dimension is fixed. By direct application of this result, we prove in Sect. 4.5 that the Pooling Problem with $|I| = 1$ and $|S|$ fixed is also solvable in polynomial time.

4.2 Illustration

Consider an instance with sources $S = \{s_0, s_1, s_2\}$, terminals $T = \{t_1, t_2\}$, and $|K| = 2$. Let the source quality matrix be $\boldsymbol{Q} = (\boldsymbol{q}_{s_0}, \boldsymbol{q}_{s_1}, \boldsymbol{q}_{s_2}) = \begin{pmatrix} 0 & 1 & -2 \\ 0 & -2 & 1 \end{pmatrix}$ and let the quality bounds be $\boldsymbol{U} = (\boldsymbol{u}_{t_1}, \boldsymbol{u}_{t_2}) = \begin{pmatrix} 0 & \frac{1}{2} \\ -\frac{1}{2} & 0 \end{pmatrix}$. Thus, $|S_0| = 2$, and (16)–(17) define the two-dimensional unit simplex depicted in Fig. 1.

Letting k index the first row of $\boldsymbol{Q}$ and $\boldsymbol{U}$, inequality (18) becomes (for the case of brevity, write $x_{s_j} = x_j$ $(j = 1, 2)$) $x_1 - 2x_2 \leq 0$ for $t = t_1$, and $x_1 - 2x_2 \leq \frac{1}{2}$ for $t = t_2$. For $t = t_1$, the inequality is illustrated by a straight line intersecting

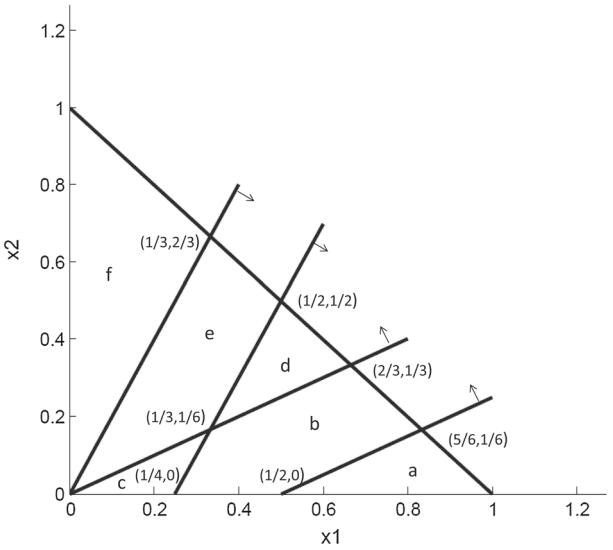


Fig. 1 Partition of the unit simplex by hyperplanes

the boundary of the simplex in Fig. 1 at points $(0, 0)$ and $\left(\frac{2}{3}, \frac{1}{3}\right)$. Analogously, for $t = t_2$, the intersection points are $\left(\frac{1}{2}, 0\right)$ and $\left(\frac{5}{6}, \frac{1}{6}\right)$. Inequalities $-2x_1 + x_2 \leq \frac{-1}{2}$ and $-2x_1 + x_2 \leq 0$, corresponding to the second row of $\boldsymbol{Q}$ and $\boldsymbol{U}$, give the remaining two lines intersecting the simplex.

The hyperplanes consisting of all $\boldsymbol{x}$-vectors satisfying one of the constraints (18) with equality thus partition the unit simplex into polytopes denoted a, b, c, d, e and f. We observe that polytope d corresponds to $T^+ = \{t_1, t_2\}$, polytopes b, c and e correspond to $T^+ = \{t_2\}$, and polytopes a and f correspond to $T^+ = \emptyset$. Thus, $\mathcal{T}(\mathcal{D}, \boldsymbol{b}) = \{T, \{t_2\}, \emptyset\}$, which means that the LP (11)–(14) does not have to be solved for $T^+ = \{t_1\}$.

In Fig. 1, there are 10 points appearing as a vertex in at least one of the polytopes. These are (1/3, 1/6) and 9 points on the boundary of the simplex. Each vertex point is identified by the solution to a system of linear equations. A pair of vertex points is further an edge of some polytope if and only if there exist $n - 1$ distinct hyperplanes corresponding to (16)–(18) intersecting both points, and no such hyperplane separating the points. We observe that in Fig. 1, $[(0, 0), (1/3, 1/6)]$ is an edge of c and e. Due to the line separating (5/6, 1/6) and (1/2, 1/2), $[(5/6, 1/6), (1/2, 1/2)]$ is not an edge of any polytope. Neither is $[(1/3, 1/6), (1/3, 2/3)]$, since there is no line corresponding to (16)–(18) intersecting (1/3, 1/6) and (1/3, 2/3).

In addition to the polytopes in the partition, also their proper faces are of interest. In cases where two or more of the hyperplanes corresponding to (18) coincide, some $T^+ \in \mathcal{T}(\mathcal{D}, \boldsymbol{b})$ can map to a polytope of dimension smaller than $|S_0|$. The algorithm to be developed next therefore produces a list of all faces of polytopes in the partition of the simplex.

4.3 Polyhedral Partitions of the Unit Simplex

4.3.1 Definitions

Let $P \subset \mathbb{R}^n$ denote a polytope and $V(P)$ its vertex set. For a finite set $V \subset \mathbb{R}^n$, let $P(V)$ denote the polytope $\mathrm{conv}V$. A polytope of dimension n is referred to as an n-polytope, and analogous nomenclature is used for other geometrical structures in vector spaces.

Define the index sets $M = \{1, \ldots, m+n+1\}$ and $N = \{1, \ldots, n\}$, and consider a matrix $\boldsymbol{A} \in \mathbb{Q}^{m\times n}$ and a vector $\boldsymbol{d} \in \mathbb{Q}^m$. Associate with $(\boldsymbol{A}, \boldsymbol{d})$ the extension $(\bar{\boldsymbol{A}}, \bar{\boldsymbol{d}})$, where

$$\bar{\boldsymbol{A}} = \begin{pmatrix} \boldsymbol{A} \\ \boldsymbol{I}^n \\ 1 \ \cdots \ 1 \end{pmatrix} = \left(a_{ij}\right)_{i\in M, j\in N}, \ \bar{\boldsymbol{d}} = \begin{pmatrix} \boldsymbol{d} \\ \boldsymbol{0}^n \\ 1 \end{pmatrix} = (d_i)_{i\in M}.$$

Observe that the last $n+1$ rows of $(\bar{\boldsymbol{A}}, \bar{\boldsymbol{d}})$ contain the coefficients of the linear inequalities $\sum_{j=1}^n x_j \le 1$ and $x_j \ge 0$ $(j = 1, \ldots, n)$ defining the unit n-simplex Δ^n.

Definition 4.1 A pair $(M^<, M^>)$ of sets $M^< \subseteq M$ and $M^> \subseteq M$ of row indices of $(\bar{\boldsymbol{A}}, \bar{\boldsymbol{d}})$ is referred to as a *valid cover* iff

1. $M^< \cup M^> = M$ (the rows of $(\bar{\boldsymbol{A}}, \bar{\boldsymbol{d}})$ are covered),
2. $m+1, \ldots, m+n \in M^>$ ($M^>$ covers the rows corresponding to $x_1, \ldots, x_n \ge 0$), and
3. $m+n+1 \in M^<$ ($M^<$ covers the row corresponding to $\sum_{j=1}^n x_j \le 1$). □

It is not required that $M^< \cap M^>$ is empty. Conditions 2–3 of Definition 4.1 imply that the polytope consisting of all $\boldsymbol{x} \in \mathbb{R}^n$ satisfying

$$\sum_{j=1}^n a_{ij}x_j \le d_i \ \ i \in M^<, \tag{19}$$

$$\sum_{j=1}^n a_{ij}x_j \ge d_i \ \ i \in M^>, \tag{20}$$

is contained in Δ^n. Let $\mathcal{P}^r(\boldsymbol{A}, \boldsymbol{d})$ be the set of r-polytopes $(r = -1, 0, 1, \ldots, n)$ that correspond to valid covers, and let $\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})$ be the set of vertex sets of these polytopes. Define

$$\mathcal{P}(\boldsymbol{A}, \boldsymbol{d}) = \mathcal{P}^{-1}(\boldsymbol{A}, \boldsymbol{d}) \cup \cdots \cup \mathcal{P}^n(\boldsymbol{A}, \boldsymbol{d}) \ \text{ and}$$
$$\mathcal{V}(\boldsymbol{A}, \boldsymbol{d}) = \mathcal{V}^{-1}(\boldsymbol{A}, \boldsymbol{d}) \cup \cdots \cup \mathcal{V}^n(\boldsymbol{A}, \boldsymbol{d}).$$

Here, the convention $\dim \emptyset = -1$ is used, i.e., $\mathcal{P}^{-1}(\boldsymbol{A}, \boldsymbol{d}) = \{\emptyset\}$. All valid covers $(M^<, M^>)$ where $m+1, \ldots, m+n \in M^<$ and $m+n+1 \in M^>$ correspond to $\emptyset$, but also others may do so.

Corresponding to a polytope $P \in \mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$, there is at least one valid cover. In degenerate cases, like P containing only the intersection point of three lines in the plane,

there are more than one valid cover corresponding to P. A valid cover $(M^<, M^>)$ is said to be *maximal* iff, for some $P \in \mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$,

$$M^< = \left\{ i \in M : \sum_{j=1}^{n} a_{ij} x_j \leq d_i \ \forall x \in P \right\} \text{ and}$$

$$M^> = \left\{ i \in M : \sum_{j=1}^{n} a_{ij} x_j \geq d_i \ \forall x \in P \right\}.$$

That is, $(M^<, M^>)$ is maximal if and only if $M^<$ and $M^>$ contain the indices of *all* $i \in M$ such that $\sum_{j=1}^{n} a_{ij} x_j \leq d_i$ and $\sum_{j=1}^{n} a_{ij} x_j \geq d_i$, respectively, for all $\boldsymbol{x} \in P$. Consequently, there is a one-to-one correspondence between polytopes in $\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$ and maximal valid covers. In the example of a point intersected by three lines, the indices of all three lines occur in both $M^<$ and $M^>$ in the corresponding maximal valid cover.

Definition 4.2 [22, Ch. IV, §1.2], [23, Definition 2.1.5] A finite set $\mathcal{C}$ of polyhedra in $\mathbb{R}^n$ is a *polyhedral complex* iff

1. $F \in \mathcal{C}$ if F is a face of a polyhedron $P \in \mathcal{C}$, and
2. $P_0 \cap P_1$ is a face of both P_0 and P_1 if $P_0, P_1 \in \mathcal{C}$. □

To prove that the size of $\mathcal{V}(\boldsymbol{A}, \boldsymbol{d})$ and $\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$ is polynomial in m, the concept of a polyhedral complex is instrumental.

4.3.2 Preliminary Results

Proposition 4.1 *$\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$ is a polyhedral complex.*

Proof To prove that $\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$ satisfies the first condition of Definition 4.2, consider any $P \in \mathcal{P}^r(\boldsymbol{A}, \boldsymbol{d})$, and let F be an r'-face of P, where $0 \leq r' < r \leq n$. Then there exists a valid cover $(M^<, M^>)$ for which P is the solution set of (19), (20). Because F is an r'-face of P, there also exist $r - r'$ linearly independent inequalities in (19), (20), that are satisfied with equality for all $\boldsymbol{x} \in F$, and satisfied with strict inequality for all $\boldsymbol{x} \in P \setminus F \neq \emptyset$. Let $L^< \subseteq M^<$ and $L^> \subseteq M^>$ ($|L^<| + |L^>| = r - r'$) be the indices of the inequalities meeting these criteria. Since F is the polytope corresponding to the valid cover $(M^< \cup L^>, M^> \cup L^<)$, it follows that $F \in \mathcal{P}^{r'}(\boldsymbol{A}, \boldsymbol{d}) \subset \mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$.

To prove the second condition, consider any $P_0, P_1 \in \mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$ with corresponding maximal covers $\left(M_0^<, M_0^>\right)$ and $\left(M_1^<, M_1^>\right)$, respectively. Thus, $P_0 \cap P_1$ consists of all $\boldsymbol{x} \in \mathbb{R}^n$ satisfying $\sum_{j=1}^{n} a_{ij} x_j \leq d_i \ \left(i \in M_0^< \cup M_1^<\right)$ and $\sum_{j=1}^{n} a_{ij} x_j \geq d_i \ \left(i \in M_0^> \cup M_1^>\right)$. Let $M^= = \left(M_0^< \cap M_1^>\right) \cup \left(M_1^< \cap M_0^>\right)$ and $H = \left\{ \boldsymbol{x} \in \mathbb{R}^n : \sum_{j=1}^{n} a_{ij} x_j = d_i, i \in M^= \right\}$. For $k = 0, 1$, it follows that

$$P_k \cap H = \left\{ \boldsymbol{x} \in \mathbb{R}^n : \sum_{j=1}^{n} a_{ij} x_j \leq d_i \left(i \in \widehat{M}_k^<\right), \sum_{j=1}^{n} a_{ij} x_j \geq d_i \left(i \in \widehat{M}_k^>\right) \right\},$$

where

$$\widehat{M}_k^< = M_k^< \cup M^= = M_k^< \cup \left(M_k^< \cap M_{1-k}^>\right) \cup \left(M_{1-k}^< \cap M_k^>\right) = M_k^< \cup M_{1-k}^<.$$

The last equality is obtained by exploiting $M_k^< \cup M_k^> = M$, which is true since $\left(M_k^<, M_k^>\right)$ is a valid cover. Analogously, $\widehat{M}_k^> = M_k^> \cup M_{1-k}^>$, and hence, $P_k \cap H = P_0 \cap P_1$. Because P_k is entirely contained in one of the two closed half-spaces bounded by H, it follows that $P_0 \cap P_1$ is a face of P_k. □

Proposition 4.2 *For all* $r = 0, 1, \ldots, n$, $\left|\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})\right| \leq \binom{\left|\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})\right|}{r+1}$.

Proof Let $p_r = |\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})|$, and $\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d}) = \left\{V_1, \ldots, V_{p_r}\right\}$. For all $k = 1, \ldots, p_r$, $P(V_k)$ has a simplicial decomposition [24], i.e., there exist $t_k \geq 1$ subsets $U_{k1}, \ldots, U_{kt_k} \subseteq V_k$ with $|U_{k1}| = \cdots = \left|U_{kt_k}\right| = r + 1$ such that $P(V_k)$ is covered by the set of r-simplices $P(U_{k1}), \ldots, P\left(U_{kt_k}\right)$. By Proposition 4.1, $U_{k1} \ldots, U_{kt_k}$ consist of vertices from $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$.

Consider two polytopes $P(V_k), P(V_\ell) \in \mathcal{P}^r(\boldsymbol{A}, \boldsymbol{d})$, where $V_k \neq V_\ell$. Assume there exist some $j[k] \in \{1, \ldots, t_k\}$ and $j[\ell] \in \{1, \ldots, t_\ell\}$ such that $U_{k,j[k]} = U_{\ell,j[\ell]} =: U$. Because Proposition 4.1 implies that $P(V_k) \cap P(V_\ell)$ is a face of both $P(V_k)$ and $P(V_\ell)$, we have

$$r = \dim P(U) \leq \dim\left(P(V_k) \cap P(V_\ell)\right) \leq \dim P(V_k) = \dim P(V_\ell) = r.$$

Hence, $P(V_k) \cap P(V_\ell)$ equals $P(V_k)$ and $P(V_\ell)$, contradicting $V_k \neq V_\ell$. It follows that $U_{11}, \ldots, U_{p_r 1}$ are distinct sets consisting of $r + 1$ points in $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$.

As there exist exactly $\binom{\left|\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})\right|}{r+1}$ such sets, the proof is complete. □

4.4 Enumerating all Elements in the Polyhedral Complex

Enumeration of all faces of a polyhedron is an interesting problem that has received considerable attention [25,26] in computational geometry studies. In this section, an algorithm for the following more general problem is analyzed:

Problem 4.1 Enumeration of a Polyhedral Complex:
Given $\boldsymbol{A} \in \mathbb{Q}^{m \times n}$ and $\boldsymbol{d} \in \mathbb{Q}^m$ generate a list of all vertex sets of polytopes in the polyhedral complex $\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$. □

Below, it is formally proved that Algorithm 1 produces a list of all polytopes in the polyhedral complex $\mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$. Each such polytope, $P \in \mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$, is represented by its vertex set $V(P)$. First, the list $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$ consisting of singletons is generated by selecting sets $L \subseteq M$ indexing n linearly independent rows of $\bar{\boldsymbol{A}}$. Denote the indexed rows of $\left(\bar{\boldsymbol{A}}, \bar{\boldsymbol{d}}\right)$ by $\left(\bar{\boldsymbol{A}}_L, \bar{\boldsymbol{d}}_L\right)$. Solutions to the corresponding linear systems that are contained on the unit simplex are stored in $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$, whereas others are discarded.

Next, for increasing values of r, all vertex sets in $\mathcal{V}^{r+1}(\boldsymbol{A}, \boldsymbol{d})$ are found by extending vertex sets of $\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})$. For each $U \in \mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})$ and $W \in \mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$, let V consist of

the vertices in $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$ that satisfy all inequalities in (19), (20) satisfied by $U \cup W$. Thus, $V \supseteq U \cup W$. If the inequalities satisfied by V constitute a valid cover, and $\dim P(V) = r + 1$, V is added to $\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})$.

Algorithm 1 `findPolyhedralComplex`$(\boldsymbol{A}, \boldsymbol{d})$

$\mathcal{V}^0 \leftarrow \emptyset$
for all sets $L \subset M$ indexing n linearly independent rows of $\bar{\boldsymbol{A}}$ **do**
  $\boldsymbol{x} \leftarrow \bar{\boldsymbol{A}}_L^{-1} \bar{\boldsymbol{d}}_L$, $V \leftarrow \{\boldsymbol{x}\}$
  if $\boldsymbol{x} \in \Delta^n$ **and** $V \notin \mathcal{V}^0$ **then**
    $\mathcal{V}^0 \leftarrow \mathcal{V}^0 \cup \{V\}$,
    $M_V^< \leftarrow \left\{i \in M : \sum_{j=1}^n a_{ij} x_j \le d_i\right\}$, $M_V^> \leftarrow \left\{i \in M : \sum_{j=1}^n a_{ij} x_j \ge d_i\right\}$
{Now $\mathcal{V}^0$ contains all singular point sets}
for $r \leftarrow 0, \ldots, n-1$ **do**
  $\mathcal{V}^{r+1} \leftarrow \emptyset$ {No $(r+1)$-polytope found yet}
  for all $U \in \mathcal{V}^r$ and $W \in \mathcal{V}^0$ where $W \not\subseteq U$ **do**
    $M^< \leftarrow M_U^< \cap M_W^<$, $M^> \leftarrow M_U^> \cap M_W^>$
    if $M^< \cup M^> = M$ **then** {Valid cover, no $(\bar{\boldsymbol{A}}, \bar{\boldsymbol{d}})$-hyperplane separates W from U}
      $V \leftarrow \left\{\boldsymbol{x} \in \mathcal{V}^0 : \sum_{j=1}^n a_{ij} x_j \le b_i (i \in M^<), \sum_{j=1}^n a_{ij} x_j \ge b_i (i \in M^>)\right\}$
      if $\dim P(V) = r + 1$ **then** {consider V only if $V \in \mathcal{V}^{r+1}(\boldsymbol{A}, \boldsymbol{d})$}
        $\mathcal{V}^{r+1} \leftarrow \mathcal{V}^{r+1} \cup \{V\}$, $M_V^< \leftarrow M^<$, $M_V^> \leftarrow M^>$ {an $(r+1)$-polytope is found}
return $\left(\mathcal{V}^0, \ldots, \mathcal{V}^n\right)$

For an illustration of Algorithm 1, assume that $\boldsymbol{A}$ and $\boldsymbol{d}$ correspond to the example depicted in Fig. 1. Consider the iterations of the second and third loops of the algorithm, in which r is assigned the value 0, while U and W are assigned values $\{(1/3, 1/6)\}$ and $\{(1/2, 1/2)\}$, respectively. There is no hyperplane separating the two points, and a valid cover is identified. Only points on the line segment $[(1/3, 1/6), (1/2, 1/2)]$ satisfy all inequalities satisfied by $U \cup W$, and thus, V becomes $\{(1/3, 1/6), (1/2, 1/2)\}$. Since also $\dim P(V) = 1$, V is added to $\mathcal{V}^1$. When $U = \{(5/6, 1/6)\}$ and $W = \{(1/2, 1/2)\}$, however, the hyperplane intersecting (0,0) and (2/3,1/3) separates U and W. As no valid cover is found, the condition of the first `if`-statement inside the last loop does not hold. When $U = \{(1/3, 1/6)\}$ and $W = \{(1/3, 2/3)\}$, the condition holds, and the value $\{(0, 0), (1/3, 1/6), (1/2, 1/2), (1/3, 2/3)\}$ is assigned to V. However, $\dim P(V) = 2$, and V is not added to $\mathcal{V}^1$. In the next iteration of the loop on r, U will in some iteration of the innermost loop be assigned the value $\{(1/3, 1/6), (1/2, 1/2)\}$ while $W = \{(1/3, 2/3)\}$. Then, V again becomes $\{(0, 0), (1/3, 1/6), (1/2, 1/2), (1/3, 2/3)\}$, which finally is added to $\mathcal{V}^2$.

The following result shows that Algorithm 1 generates the desired lists of vertex sets of polytopes in the polyhedral complex.

Lemma 4.1 *Let $r \in \{0, 1, \ldots, n\}$. After r iterations of the second `for`-loop of Algorithm 1, $\mathcal{V}^\ell = \mathcal{V}^\ell(\boldsymbol{A}, \boldsymbol{d})$ for all $\ell \in \{0, 1, \ldots, r\}$. For all $V \in \mathcal{V}^\ell$, $\left(M_V^<, M_V^>\right)$ is the maximal valid cover corresponding to $P(V)$.*

Proof The proof is by induction in r.

Vertex set $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$ consists of all singleton sets $\{\boldsymbol{x}\}$ such that $\boldsymbol{x} \in \Delta^n$ is the unique point satisfying n of the equations in the system $\bar{\boldsymbol{A}}\boldsymbol{x} = \bar{\boldsymbol{d}}$. This means that the first loop generates $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$, and the claims hold for $r = 0$.

To prove the induction step, choose $r \in \{0, 1, \ldots, n-1\}$, and assume the claims hold for r. The induction hypothesis says that $\left(M_U^<, M_U^>\right)$ and $\left(M_W^<, M_W^>\right)$ are the maximal valid covers corresponding to $P(U) \in \mathcal{P}^r(\boldsymbol{A}, \boldsymbol{d})$ and $P(W) \in \mathcal{P}^0(\boldsymbol{A}, \boldsymbol{d})$, respectively. If $(M^<, M^>) = \left(M_U^< \cap M_W^<, M_U^> \cap M_W^>\right)$ is valid, it is the maximal valid cover corresponding to a polytope $P \in \mathcal{P}(\boldsymbol{A}, \boldsymbol{d})$ containing $U \cup W$. Because the value assigned to V is the set of vertices in $\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})$ satisfying (19)–(20), it follows that V is the vertex set of P. If $\dim P = r + 1$, V is added to $\mathcal{V}^{r+1}$, and the corresponding maximal valid cover is assigned to $\left(M_V^<, M_V^>\right)$. Consequently, every iteration of the innermost `for`-loop leaves $\mathcal{V}^{r+1}$ unchanged or extends it by the vertex set of an $(r+1)$-polytope.

To complete the proof, we must show that every vertex set $V \in \mathcal{V}^{r+1}(\boldsymbol{A}, \boldsymbol{d})$ is added to $\mathcal{V}^{r+1}$. By the selection of U and W, it follows from Proposition 4.1 that in some iteration, U is the vertex set of a facet of $P(V)$, while $W \nsubseteq U$ is a vertex of $P(V)$. Because $P(V)$ is the unique $(r+1)$-polytope in $\mathcal{P}^{r+1}(\boldsymbol{A}, \boldsymbol{d})$ containing U and W, it follows that V will be included in $\mathcal{V}^{r+1}$. □

Proposition 4.3 *Algorithm 1 solves Problem 4.1, and its running time is polynomial in m.*

Proof Correctness of the output of Algorithm 1 follows directly from Lemma 4.1. The first `for`-loop amounts to solving $\binom{m+n+1}{n}$ systems of n linear equations, and checking $2(m+1)$ linear inequalities. Consequently, the loop runs in time bounded by a polynomial in m.

By Lemma 4.1, the number of sets V considered in iteration r of the second `for`-loop is no more than $\left|\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})\right| \cdot |\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})|$. Because $\left|\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})\right| \leq \binom{m+n+1}{n}$ and $\left|\mathcal{V}^r(\boldsymbol{A}, \boldsymbol{d})\right| \leq \binom{\left|\mathcal{V}^0(\boldsymbol{A}, \boldsymbol{d})\right|}{r+1}$ according to Proposition 4.2, the number of executions of the third `for`-loop is polynomial in m. Finally, the dimensionality check of $P(V)$ is accomplished by searching V for $r+2$ affinely independent vertices. Thus, the number of arithmetic operations inside the loops is polynomial in m, which completes the proof. □

4.5 A Polynomial Algorithm for the Pooling Problem with a Single Pool and Fixed Source Cardinality

By virtue of Algorithm 1 and Proposition 4.3, it is provable that problem (11)–(15) is solvable in polynomial time for bounded $|S|$. Let $\boldsymbol{A}$ and $\boldsymbol{d}$ have one row for each $(t, k) \in T \times K$, and one column for each $s \in S_0$. The entry in $\boldsymbol{d}$ corresponding to (t, k) is set to u_t^k, and a_{ij} is put equal to q_s^k, where i and j are the row and column corresponding to (t, k) and s, respectively. When receiving $\boldsymbol{Q}$ and $\boldsymbol{U}$ as input, Algorithm 2 outputs the set $\mathcal{T}(\mathcal{D}, \boldsymbol{b})$ of terminal sets that a feasible flow in $\mathcal{F}(\mathcal{D}, \boldsymbol{b})$ possibly can satisfy.

Theorem 4.1 *For any $s_{\max} \geq 1$, the instance class of the* Pooling Problem *with $|I| = 1$ and $|S| \leq s_{\max}$ is solvable in polynomial time.*

Proof According to Proposition 3.2, the optimal cost in all instances in question is $\min\left\{z\left(T^+\right) : T^+ \in \mathcal{T}\right\}$, where $\mathcal{T}$ is the output from Algorithm 2. Because $z\left(T^+\right)$ is the solution to the compact LP (11)–(14), the claim then follows directly from Proposition 4.3 with $n = |S| - 1 < s_{\max}$. □

Corollary 4.1 *The extension of Problem 2.1 where a positive lower bound is imposed on the flow entering each terminal in $T' \subseteq T$ is solvable in polynomial time for instances where $|I| = 1$ and $|S| \leq s_{\max}$.*

Algorithm 2 `findTerminals`$(\boldsymbol{Q}, \boldsymbol{U})$

Define the matrix $\boldsymbol{A} \in \mathbb{Q}^{(|T\times K|)\times|S_0|}$ and the vector $\boldsymbol{d} \in \mathbb{Q}^{|T\times K|}$ such that they correspond to inequalities (18). Let $n \leftarrow |S_0|$.
Find $\mathcal{V}^0, \ldots, \mathcal{V}^n$ by calling `findPolytopes`$(\boldsymbol{A}, \boldsymbol{d})$
$\mathcal{T} \leftarrow \{T\}$ {all terminals are satisfied by $\mathbf{0}^{|\mathcal{A}|}$}
for $r \leftarrow 0, \ldots, n$ **do** {loop through all polytope dimensions r}
  for all $V \in \mathcal{V}^r$ **do** {loop through all r-polytopes}
    $\boldsymbol{x} \leftarrow \dfrac{1}{|V|}\displaystyle\sum_{\boldsymbol{y}\in V} \boldsymbol{y}$ {let $\boldsymbol{x}$ be any relative interior point of $P(V)$}
    $T^+ \leftarrow T$ {assume $\boldsymbol{x}$ satisfies all quality bounds of all terminals}
    for all $k \in K$ **do** {check quality parameter k}
      $T^+ \leftarrow \left\{t \in T^+ : \displaystyle\sum_{s\in S_0} q_s^k x_s \leq u_t^k\right\}$ {keep only terminals that $\boldsymbol{x}$ satisfies}
    $\mathcal{T} \leftarrow \mathcal{T} \cup \{T^+\}$ {include the feasible terminal set}
return $\mathcal{T}$

Proof Follows immediately by modifying Algorithm 2 such that all terminal sets $T^+ \in \mathcal{T}(\mathcal{D}, \boldsymbol{b})$ that do not contain T' are discarded. □

It is also straightforward to adapt the proofs discussed in Sects. 3.2–3.3 to the extension considered in Corollary 4.1. In conclusion, the extended problem is solvable in polynomial time for the single-pool instance classes where either $|T|$, $|K|$ or $|S|$ is bounded by a constant.

Recall from Sect. 2 that $\mathcal{D}$ is defined to have arcs only from S to I and from I to T. Given this restriction, instances that accept direct flow from a source s to a terminal t contain a pool i where $S_i = \{s\}$ and $T_i = \{t\}$. Alternatively, $\mathcal{D}$ could be defined such that the direct arc from s to t is allowed, making the pool i redundant. Most studies [3,10,15,20] follow this convention. It is, however, important to note that the results in Sects. 3–4 are not proved to hold if $|I| = 1$ in this alternative definition of $\mathcal{D}$. That is, it is an open question whether the Pooling Problem is solvable in polynomial time when either $|T|$, $|K|$ or $|S|$ is fixed, and only one pool has more than one entering arc and more than one leaving arc.

Remark 4.1 After submission of the current text, Boland et al. [27] submitted a manuscript with an alternative proof of Proposition 4.1. Their analysis has points in common with the analysis in Sect. 4.3, as it also considers partitions of the unit simplex induced

by a set of linear inequalities. Moreover, [27] proves that in instances where $|I| = 1$, the POOLING PROBLEM is solvable in terms of at most $\sum_{j=0}^{|S|-1} \binom{|K|}{j} |T|^j$ compact linear programs. □

5 The Pooling Problem with a Single Quality Parameter

In the remainder of the text, we study a rich instance class where $|I|$ is arbitrary, $|K| = 1$, and $|S| = |T| = 2$. By a reduction from EXACT COVER BY 3-SETS, it is proved [21] that the POOLING PROBLEM remains strongly NP-hard when $|K| = 1$. When also $|S| = |T| = 2$, at least weak NP-hardness persists. However, the reduction [21] from PACKING IN TWO BINS for the $|S| = |T| = 2$ case does not prove strong NP-hardness. PACKING IN TWO BINS asks whether a set of items of given integer weights fit into two bins with identical integer capacities. There exists a dynamic programming algorithm, which runs in polynomial time for instance classes where item weights and bin capacities are bounded by some polynomial function of the input size. In this section, we show how the dynamic programming algorithm for PACKING IN TWO BINS can be adapted to the instance class of the POOLING PROBLEM under study.

5.1 A Dynamic Programming Algorithm

Each recursive step of the dynamic programming algorithm for PACKING IN TWO BINS amounts to selecting the bin into which a given item should be put. The corresponding operation in the algorithm given below is to select in which out of five possible subsets a given pool belongs. Each of four of the subsets corresponds to a particular source or terminal, and all pools allocated to the set receive (send) zero flow from (to) the source (terminal) in question. Pools allocated to the fifth set receive (send) positive flow from (to) both sources (terminals).

It is shown below that the collection of pools in one of the former four sets can be replaced by a new pool with capacity equal to the total capacity in the set. Further, it is shown that under the assumptions made, it is optimal to have at most one pool in the latter set. Hence, the dynamic programming algorithm reduces the network to an instance with only five pools. Instances of this small, fixed size are shown to have closed-form solutions, and they are thereby solvable in constant time. Since a running time analysis of the dynamic programming algorithm proves that the reduction runs in pseudo-polynomial time, we thus have an algorithm with pseudo-polynomial running time for $|K| = 1$ and $|S| = |T| = 2$.

Because $|K| = 1$, indices corresponding to quality parameters are omitted in all notation used. Also for the purpose of simplified notation, we associate the nodes $\mathcal{N}$ with integers, and for reasons to become clear later, we let $S = \{0, 1\}$ and $T = \{7, 8\}$. Without loss of generality [21, Proposition 2], we assume $q_0 = 0$, $q_1 = 1$, and $u_7 \leq u_8$. Further, we focus on instances with pool independent costs, i.e., all arcs leaving (entering) the same source (terminal) have identical costs. We denote by c_s and c_t the unit costs of arcs $(s, i) \in \mathcal{A}_S$ and $(i, t) \in \mathcal{A}_T$, respectively. For any feasible

flow, we define the *flow degree* of a node as the number of incident arcs carrying positive flow.

Proposition 5.1 POOLING PROBLEM *instances with* $|K| = 1$, $|S| = |T| = 2$, *and pool independent costs have optimal solutions where at most one pool has flow degree 4.*

Proof Let $\boldsymbol{f}$ be an optimal flow, and assume $f_{si}, f_{sj}, f_{it}, f_{jt} > 0$ for all $s \in S$ and $t \in T$, where $i, j \in I$ and $i \neq j$. Consider the cycles $C_1 = (1, j, 0, i, 1)$ and $C_2 = (j, 8, i, 7, j)$ consisting of forward and backward arcs in $\mathcal{D}$. Flow augmentations ε and δ along cycles C_1 and C_2, respectively, yield a flow denoted $\boldsymbol{g}$, where $\boldsymbol{g} = \boldsymbol{f}$ if $\varepsilon = \delta = 0$ (the dependency of ε and δ in $\boldsymbol{g}$ is suppressed for the purpose of simplified notation). Let Ω be the set of pairs (ε, δ) for which $\boldsymbol{g} \geq \boldsymbol{0}^{|\mathcal{A}|}$. That is, Ω consists of all (ε, δ) satisfying

$$
\begin{aligned}
&-\min\left\{f_{0i}, f_{1j}\right\} \leq \varepsilon \leq \min\left\{f_{0j}, f_{1i}\right\}, \text{ and}\\
&-\min\left\{f_{i7}, f_{j8}\right\} \leq \delta \leq \min\left\{f_{j7}, f_{i8}\right\}.
\end{aligned}
$$

Because the augmentations do not affect the capacity consumption at the nodes, $\boldsymbol{g} \in \mathcal{F}(\mathcal{D}, \boldsymbol{b})$ if $(\varepsilon, \delta) \in \Omega$. Since the costs are pool independent, also the objective function values at $\boldsymbol{f}$ and $\boldsymbol{g}$ are identical. It follows that $\boldsymbol{g}$ is optimal if it satisfies the quality bounds.

Let $\boldsymbol{w} \in \mathbb{R}^{|\mathcal{N}|}$ and $\boldsymbol{v} \in \mathbb{R}^{|\mathcal{N}|}$ be the quality vectors corresponding to $\boldsymbol{f}$ and $\boldsymbol{g}$, respectively. We prove the proposition by showing that there exists an augmentation $(\varepsilon, \delta) \in \Omega$ such that $\boldsymbol{g}$ is zero on one arc of C_1 or C_2, and such that $v_7 = w_7$ and $v_8 = w_8$.

Let $F_\ell = f_{\ell 7} + f_{\ell 8} = g_{\ell 7} + g_{\ell 8}$ be the total flow through pool $\ell \in I$. From $q_0 = 0$ and $q_1 = 1$, it follows that $w_i = f_{1i}/F_i$ and $w_j = f_{1j}/F_j$. We also have

$$
\begin{aligned}
&v_i = \frac{w_i F_i - \varepsilon}{F_i} = \frac{f_{1i} - \varepsilon}{F_i}, \text{ and } v_j = \frac{w_j F_j + \varepsilon}{F_j} = \frac{f_{1j} + \varepsilon}{F_j}, \text{ yielding}\\
&v_7 = \frac{w_7 F_7 - w_i f_{i7} - w_j f_{j7} + v_i g_{i7} + v_j g_{j7}}{F_7} =\\
&w_7 + \frac{-f_{1i} f_{i7}/F_i - f_{1j} f_{j7}/F_j + (f_{1i} - \varepsilon)(f_{i7} + \delta)/F_i + (f_{1j} + \varepsilon)(f_{j7} - \delta)/F_j}{F_7}.
\end{aligned}
$$

It follows that $v_7 = w_7$ if

$$
\varepsilon\delta\left(F_i + F_j\right) + \varepsilon\left(F_j f_{i7} - F_i f_{j7}\right) + \delta\left(F_i f_{1j} - F_j f_{1i}\right) = 0. \tag{21}
$$

The set of pairs (ε, δ) satisfying (21) is the union of two continuous curves H_1 and H_2. If $F_j f_{i7} - F_i f_{j7} \neq 0 \neq F_i f_{1j} - F_j f_{1i}$, then $H_1 \cup H_2$ is a hyperbola. Otherwise, $H_1 \cup H_2$ consists of two straight lines, one of which is a coordinate axis (where either $\varepsilon = 0$ or $\delta = 0$). Because neither $H_1 \setminus \Omega$ nor $H_2 \setminus \Omega$ is empty, and $(0, 0) \in (H_1 \cup H_2) \cap \Omega$, continuity implies that $H_1 \cup H_2$ intersects the boundary of Ω. It follows that there exists some (ε, δ) such that $\boldsymbol{g}$ is zero on one of the arcs of C_1 or C_2, and such that $v_7 = w_7$.

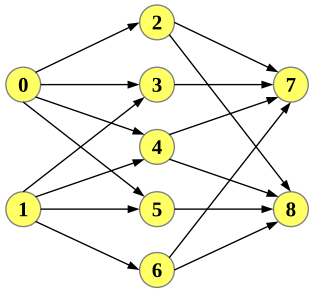


Fig. 2 Atomic instance

Because $\sum_{\ell\in I_s} f_{s\ell} = \sum_{\ell\in I_s} g_{s\ell}$ $(s \in S)$ and $\sum_{\ell\in I_t} f_{\ell t} = \sum_{\ell\in I_t} g_{\ell t}$ $(t \in T)$, (5) implies that also $v_8 = w_8$, which completes the proof. □

Motivated by Proposition 5.1, we search for the optimal partition of the pools I into 5 subsets denoted $J_2, \ldots, J_6$. The subset J_2 (J_6) consists of pools that do not receive flow from source 1 (source 0), and J_3 (J_5) consists of pools the do not send flow to terminal 8 (terminal 7). Subset J_4 is either empty or consists of the unique pool of flow degree 4. Ties are broken arbitrarily. Associated with the partition $(J_2, \ldots, J_6)$, we define another POOLING PROBLEM instance with directed graph $\bar{\mathcal{D}} = \left(\bar{\mathcal{N}}, \bar{\mathcal{A}}\right)$ (see Fig. 2), henceforth referred to as an *atomic* instance. The node set $\bar{\mathcal{N}} = \bar{S} \cup \bar{I} \cup \bar{T}$ consists of the sources $\bar{S} = S = \{0, 1\}$, pools $\bar{I} = \{2, \ldots, 6\}$, and terminals $\bar{T} = T = \{7, 8\}$.

Each pool $\ell \in \bar{I}$ corresponds to one of the sets $J_2, \ldots, J_6$. The arc set $\bar{\mathcal{A}}$ is hence defined such that the neighbors of sources 0 and 1 and terminals 7 and 8 are $\bar{I}_0 = \bar{I}\setminus\{6\}$, $\bar{I}_1 = \bar{I} \setminus \{2\}$, $\bar{I}_7 = \bar{I} \setminus \{5\}$, and $\bar{I}_8 = \bar{I} \setminus \{3\}$, respectively. Costs $\boldsymbol{c}$, source qualities $\boldsymbol{q}$, and terminal quality bounds $\boldsymbol{u}$ are identical to those of the original instance. The same is true for source and terminal capacities, whereas the capacity of pool $\ell \in \bar{I}$ is the cumulative capacity $B_\ell = \sum_{i\in J_\ell} b_i$ of pools in J_ℓ. Denote the node capacity vector by $\boldsymbol{B}$, where $B_s = b_s$ and $B_t = b_t$ $(s \in \bar{S}, t \in \bar{T})$. For the case of notational brevity, the dependency of $(J_2, \ldots, J_6)$ in $\boldsymbol{B}$ is suppressed.

Let $\zeta\ (\mathcal{D}, \boldsymbol{b})$ denote the minimum cost in the given instance. Relevance of the atomic instances is justified as follows:

Proposition 5.2 POOLING PROBLEM *instances with* $|K| = 1$, $|S| = |T| = 2$ *and pool independent costs, have* $\zeta\ (\mathcal{D}, \boldsymbol{b}) = \min \zeta\ \left(\bar{\mathcal{D}}, \boldsymbol{B}\right)$, *where the minimum is defined over all partitions* $(J_2, \ldots, J_6)$ *of* I *such that* $|J_4| \leq 1$.

Proof To prove $\zeta\ (\mathcal{D}, \boldsymbol{b}) \geq \min \zeta\ \left(\bar{\mathcal{D}}, \boldsymbol{B}\right)$, let $\boldsymbol{f}$ be an optimal flow in $\mathcal{D}$ with corresponding partition $(J_2, \ldots, J_6)$ of I. Without loss of generality (Proposition 5.1), assume $|J_4| \leq 1$. Let $g_{s\ell} = \sum_{i\in J_\ell} f_{si}$ $(s \in \bar{S}, \ell \in \bar{I}_s)$ and $g_{\ell t} = \sum_{i\in J_\ell} f_{it}$ $(t \in \bar{T}, \ell \in \bar{I}_t)$. Because $B_\ell = \sum_{i\in J_\ell} b_i$ and $\boldsymbol{f} \in \mathcal{F}\ (\mathcal{D}, \boldsymbol{b})$, it follows immediately that $g \in \mathcal{F}\ \left(\bar{\mathcal{D}}, \boldsymbol{B}\right)$ and

$$\sum_{s\in\bar{S}} c_s \sum_{\ell\in\bar{I}_s} g_{s\ell} + \sum_{t\in\bar{T}} c_t \sum_{\ell\in\bar{I}_t} g_{\ell t} = \sum_{(s,i)\in\mathcal{A}_S} c_s f_{si} + \sum_{(i,t)\in\mathcal{A}_T} c_t f_{it}.$$

It only remains to prove that $\boldsymbol{g}$ satisfies the quality constraints. Let $\boldsymbol{w}$ and $\boldsymbol{v}$ be quality vectors corresponding to $\boldsymbol{f}$ and $\boldsymbol{g}$, respectively. Thus, $v_2 = w_i = 0$ $(i \in J_2)$ and $v_6 = w_i = 1$ $(i \in J_6)$. Pool quality v_ℓ is a convex combination of $\{w_i\}_{i\in J_\ell}$ $(\ell = 3, 5)$, given by $v_3 = \sum_{i\in J_3} w_i \frac{f_{i7}}{g_{37}}$ and $v_5 = \sum_{i\in J_5} w_i \frac{f_{i8}}{g_{58}}$. With respect to the quality v_4, if $|J_4| = 1$, we have $g_{s4} = f_{si'}$ for $i' \in J_4$ and $s \in \bar{S}$, yielding $v_4 = w_{i'}$. Otherwise, v_4 is arbitrary. Then $\sum_{\ell\in\bar{I}_7} v_\ell g_{\ell 7}$ equals

$$v_3 g_{37} + v_4 g_{47} + v_6 g_{67} = \sum_{i\in J_3} w_i \frac{f_{i7}}{g_{37}} g_{37} + w_{i'} f_{i'7} + \sum_{i\in J_6} w_i f_{i7} = \sum_{i\in I_7} w_i f_{i7}.$$

It follows that

$$v_7 = \frac{\sum_{\ell\in\bar{I}_7} v_\ell g_{\ell 7}}{\sum_{\ell\in\bar{I}_7} g_{\ell 7}} = \frac{\sum_{i\in I_7} w_i f_{i7}}{\sum_{i\in I_7} f_{i7}} = w_7 \le u_7,$$

since $\boldsymbol{f}$ is feasible. Fulfillment of $v_8 \le u_8$ is proved analogously.

To prove $\zeta(\mathcal{D}, \boldsymbol{b}) \le \min \zeta\left(\bar{\mathcal{D}}, \boldsymbol{B}\right)$, let $\boldsymbol{g}$ be an optimal flow in $\bar{\mathcal{D}}$. For $(s, \ell) \in \{(0, 2), (1, 6)\}$, let $f_{si} = g_{s\ell} \cdot b_i / B_\ell$ and $f_{it} = f_{si} \cdot g_{\ell t} / g_{s\ell}$ $(i \in J_\ell, t \in T)$. For $(\ell, t) \in \{(3, 7), (5, 8)\}$, let $f_{it} = g_{\ell t} \cdot b_i / B_\ell$ and $f_{si} = f_{it} \cdot g_{s\ell} / g_{\ell t}$ $(i \in J_\ell, s \in S)$. If $|J_4| = 1$, let $f_{si'} = g_{s4}$ and $f_{i't} = g_{4t}$, $(s \in S, t \in T, i' \in J_4)$. Utilizing $B_\ell = \sum_{i\in J_\ell} b_i$ and feasibility of $\boldsymbol{g}$, the proof of feasibility of $\boldsymbol{f}$ is analogous to the first part of the proof. Because the objective function values of $\boldsymbol{f}$ and $\boldsymbol{g}$ in their respective problems are equal, the proof is complete. □

Observe that $|J_4| \le 1$ is an essential condition in Proposition 5.2. If rather $|J_4| > 1$, letting $g_{s4} = \sum_{i\in J_4} f_{si}$ and $g_{4t} = \sum_{i\in J_4} f_{it}$ $(s \in S, t \in T)$ does not necessarily yield $v_t = w_t$. Aggregating pools in J_ℓ into one common pool is valid only if either their in-degree or out-degree is one. This is closely related to the fact that the POOLING PROBLEM is an LP in case $\min\{|S_i|, |T_i|\} = 1$ for all pools $i \in I$ [21, Proposition 3].

We now go on to derive a dynamic programming algorithm for reducing the original instance to an atomic instance corresponding to an optimal partition $(J_2, \ldots, J_6)$. For simplified exposition, we assume henceforth that all pool capacities b_i are integers. Denote the original pools by $i_1, \ldots, i_m$, where $m = |I|$. At stage $n = 1, \ldots, m, m+1$ of the dynamic program, each of the pools $i_1, \ldots, i_{n-1}$ has been assigned to some set J_ℓ, whereas pools $i_n, \ldots, i_m$ are left to be assigned. The state variable is the capacity vector $\boldsymbol{B}$, where B_ℓ is the total capacity currently assigned to J_ℓ.

The cost function $z_n(\boldsymbol{B})$ represents the minimum obtainable cost given state $\boldsymbol{B}$ at stage n. At stage $m + 1$, all pools are allocated, and $z_{m+1}(\boldsymbol{B})$ is the minimum cost of an atomic instance with pool capacities $\boldsymbol{B}$. The assignment of i_n made at stage $n \le m$ is represented by a unit vector in the feasible set

$$E(\boldsymbol{B}) = \left\{ \boldsymbol{e} = (e_2, \ldots, e_6) \in \{0, 1\}^5, \sum_{\ell=2}^{6} e_\ell = 1, e_4 = 0 \text{ if } B_4 > 0 \right\}.$$

Optimizing the assignment yields the backward recursion formula

$$z_n(\boldsymbol{B}) = \min_{\boldsymbol{e}\in E(\boldsymbol{B})} z_{n+1}(\boldsymbol{B} + b_n\boldsymbol{e}),$$

where $n = 1, \ldots, m$, and $z_1(0)$ equals the optimal objective function value $\zeta(\mathcal{D}, \boldsymbol{b})$. Correspondingly, the dynamic programming procedure depicted in Algorithm 3 applies.

Algorithm 3 DP(S,I,T,$\mathcal{A}$,$\boldsymbol{c}$,$\boldsymbol{b}$,$\boldsymbol{q}$,$\boldsymbol{u}$)

for $\forall \boldsymbol{B} \in \mathbb{N}^5 : \sum_{\ell=2}^{6} B_\ell = \sum_{i\in I} b_i$ **do**
  $z_{m+1}(\boldsymbol{B}) \leftarrow$ minimum cost in the atomic instance with pool capacities $\boldsymbol{B}$
for $n \leftarrow m, \ldots, 1$ **do**
  for $\forall \boldsymbol{B} \in \mathbb{N}^5 : \sum_{\ell=2}^{6} B_\ell = \sum_{j=1}^{n-1} b_{i_j}$ **do**
    $z_n(\boldsymbol{B}) = \min\left\{z_{n+1}(\boldsymbol{B} + b_n\boldsymbol{e}) : \boldsymbol{e} \in E(\boldsymbol{B})\right\}$
return $z_1(0)$

5.2 Running Time Analysis

In the first loop of Algorithm 3, $N = \binom{4+\sum_{i\in I} b_i}{4}$ atomic instances are solved. The remainder of the algorithm runs in $\mathcal{O}(N|I|)$ time. Despite its fixed size, it is not obvious that an atomic instance is solvable by a constant number of arithmetic operations. Making a reasonable assumption about the arc costs is, however, a sufficient condition:

Assumption 1 (*Quality consistent costs*) The arc costs $\boldsymbol{c}$ satisfy:

1. $c_0 > c_1$,
2. $c_8 > c_7$,
3. $c_0 + c_8 > 0$, and
4. $(1 - u_t)c_0 + u_t c_1 + c_t < 0, t = 7, 8.$ □

Assumption 1 is likely to hold in many practical instances. The first two points state, respectively, that the source supplying the better quality is also the more expensive and that the sales prices are higher at the terminal with the stricter quality requirement. Further, Assumption 1.3 states that it is not profitable to produce the poorer end product (at terminal 8) uniquely by use of the more expensive supply (source 0). Finally, the last point states that at both terminals, a blend satisfying the quality constraints by equality is profitable.

For atomic instances fulfilling Assumption 1, a solution algorithm with proven convergence in a constant number of operations on rational numbers has recently been given [28]. Assumptions 1.1–1.3 are made merely in order to focus the study and to limit the number of cases to be investigated. If Assumption 1.4 does not hold, at least one of the terminals can be removed, and the resulting POOLING PROBLEM instance becomes a linear program. Quality consistent instances do not appear to be easier than others, and we therefore conjecture that the result in [28] also applies to

instances that do not fulfill Assumption 1. The proof of [21, Theorem 3] shows that the POOLING PROBLEM with $|K| = 1$ and $|S| = |T| = 2$ is NP-hard even when the costs are pool independent and quality consistent.

Let $\nu(\mathcal{D}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{q}, \boldsymbol{u})$ denote the size of a problem instance. It follows from the running time analysis that for any polynomial $p : \mathbb{Z} \mapsto \mathbb{Z}$, there exists a solution algorithm with polynomial running time for the class of POOLING PROBLEM instances where $|K| = 1$, $|S| = |T| = 2$, costs are pool independent and quality consistent, and $\sum_{i \in I} b_i \leq p(\nu(\mathcal{D}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{q}, \boldsymbol{u}))$. We have thereby proved:

Theorem 5.1 *The* POOLING PROBLEM *with* $|K| = 1$, $|S| = |T| = 2$, *and pool independent costs satisfying Assumption* 1 *is solvable in pseudo-polynomial time.*

6 Conclusions

This work presents polynomial running time algorithms for two instance classes of the POOLING PROBLEM. In instances where the flow network contains only one pool, and an upper bound on the source cardinality applies, the problem can be solved in polynomial time by solving an associated partitioning problem: Hyperplanes corresponding to quality constraints intersect the unit simplex in the space of the source proportion variables. A list of the polytopes into which the simplex is partitioned yields a list of linear programs that can be solved in order to solve the original problem. Polynomial running time of this algorithm is formally proved by proving that the length of the list is polynomial in the number of constraints.

Instances with only one quality parameter and two sources and terminals are solved by a dynamic programming algorithm. This algorithm is based upon a reduction of general instances to instances of fixed size. When conditions that are likely to apply in practice are assumed, it is proved that the instances of constant size are solvable by a constant number of arithmetic operations. From this it is directly concluded that the dynamic programming algorithm has pseudo-polynomial running time.

The contribution offered by the two algorithms under study is mainly of a theoretical character. Questions on the computational complexity of the POOLING PROBLEM that in earlier studies were left open, are now answered. In their present form, the algorithms appear to be computationally demanding, despite their favorable theoretical properties. How, and to what extent, the algorithmic ideas presented in this work can be made suitable for solving practical instances, is left as a challenge for future research.

Applications of the algorithms can possibly be found in the design of heuristic methods for more general instance classes than those considered in the present text. Algorithms tailored for the one-pool case can be applied in order to construct feasible flows in networks with multiple pools. This follows from the obvious fact that any feasible flow using only one pool is feasible even if other pools are present in the network. Further, any convex combination of feasible flows using distinct pools is also feasible. Taking advantage of these observations, construction heuristics for multi-pool instances can be designed.

Heuristics for improving a given feasible solution can also be designed by utilizing algorithmic elements presented in this paper. The idea of flow augmenting paths [28] and cycles can be generalized to instances with more than one quality parameter.

Improvement heuristics for general instances can thus be based upon identification of subnetworks in which the flow can be augmented, such that costs are reduced and feasibility is preserved. Exploiting these ideas in order to find computationally efficient solution methods is left as a challenge for future research.

Acknowledgments The work of the second author has been funded by grants from the Spanish state (TIN2015-66680), Junta de Andalucía (P11-TIC-7176), in part financed by the European Regional Development Fund (ERDF).

References

1. Stigler, G.J.: The cost of subsistence. J. Farm Econ. **27**(2), 303–314 (1945)
2. Dantzig, G.B.: The diet problem. Interfaces **20**(4), 43–47 (1990)
3. Haverly, C.A.: Studies of the behavior of recursion for the pooling problem. ACM SIGMAP Bull. **25**, 19–28 (1978)
4. Baker, T.E., Lasdon, L.S.: Successive linear programming at Exxon. Manag. Sci. **31**(3), 264–274 (1985)
5. DeWitt, C.W., Lasdon, L.S., Waren, A.D., Brenner, D.A., Melham, S.: OMEGA: an improved gasoline blending system for Texaco. Interfaces **19**(1), 85–101 (1989)
6. Floudas, C.A., Visweswaran, V.: A global optimization algorithm (GOP) for certain classes of nonconvex NLPs: I. Theory. Comput. Chem. Eng. **14**(12), 1397–1417 (1990)
7. Audet, C., Brimberg, J., Hansen, P., Le Digabel, S., Mladenović, N.: Pooling problem: alternate formulations and solution methods. Manag. Sci. **50**(6), 761–776 (2004)
8. Foulds, L.R., Haugland, D., Jörnsten, K.: A bilinear approach to the pooling problem. Optimization **24**, 165–180 (1992)
9. Misener, R., Thompson, J.P., Floudas, C.A.: APOGEE: global optimization of standard, generalized, and extended pooling problems via linear and logarithmic partitioning schemes. Comput. Chem. Eng. **35**, 876–892 (2011)
10. Sahinidis, N.V., Tawarmalani, M.: Accelerating branch-and-bound through a modeling language construct for relaxation-specific constraints. J. Glob. Optim. **32**(2), 259–280 (2005)
11. Visweswaran, V., Floudas, C.A.: Computational results for an efficient implementation of the GOP algorithm and its variants. In: Grossmann, I.E. (ed.) Global Optimization in Chemical Engineering, pp. 111–153. Kluwer Academic Publishers, Dordrecht (1996)
12. Adhya, N., Tawarmalani, M., Sahinidis, N.V.: A Lagrangian approach to the pooling problem. Ind. Eng. Chem. Res. **38**(5), 1965–1972 (1999)
13. Almutairi, H., Elhedhli, S.: A new Lagrangian approach to the pooling problem. J. Glob. Optim. **45**(2), 237–257 (2009)
14. Ben-Tal, A., Eiger, G., Gershovitz, V.: Global minimization by reducing the duality gap. Math. Program. **63**(2), 193–212 (1994)
15. Dey, S.S., Gupte, A.: Analysis of MILP techniques for the pooling problem. Oper. Res. **62**(2), 412–427 (2015)
16. Kallrath, J.: Solving planning and design problems in the process industry using mixed integer and global optimization. Ann. Oper. Res. **140**(1), 339–373 (2005)
17. Galan, B., Grossmann, I.E.: Optimal design of distributed wastewater treatment networks. Ind. Eng. Chem. Res. **37**(10), 4036–4048 (1998)
18. Misener, R., Floudas, C.A.: Global optimization of large-scale generalized pooling problems: quadratically constrained MINLP models. Ind. Eng. Chem. Res. **49**, 5424–5438 (2010)
19. Misener, R., Floudas, C.A.: Advances for the pooling problem: modeling, global optimization, and computational studies. Appl. Comput. Math. **8**, 3–22 (2009)
20. Alfaki, M., Haugland, D.: Strong formulations for the pooling problem. J. Glob. Optim. **56**(3), 897–916 (2013)
21. Haugland, D.: The computational complexity of the pooling problem. J. Glob. Optim. **64**(2), 199–215 (2016)
22. Aleksandrov, P.S.: Combinatorial Topology, vol. 1. Graylock Press, Rochester, New York (1956)

23. De Loera, J.A., Rambau, J., Santos, F.: Triangulations: Structures for Algorithms and Applications. Springer, Berlin (2010)
24. Edmonds, A.L.: Simplicial decompositions of convex polytopes. Pi Mu Epsil. J. **5**(3), 124–128 (1970)
25. Murty, K.G., Chung, S.-J.: Segments in enumerating faces. Math. Program. **70**(1), 27–45 (1995)
26. Fukuda, K., Liebling, T.M., Margot, F.: Analysis of backtrack algorithms for listing all vertices and all faces of a convex polyhedron. Comput. Geom. Theor. Appl. **8**, 1–12 (1997)
27. Boland, N., Kalinowski, T., Rigterink, F.: A polynomially solvable case of the pooling problem. arXiv:1508.03181v2 (math.OC) (2015)
28. Haugland, D., Hendrix, E.M.T.: On a pooling problem with fixed network size. In: Corman, F., Voß, S., Negenborn, R.R. (eds.) Proceedings of the 6th International Conference on Computational Logistics, ICCL 2015, Lecture Notes in Computer Science, vol. 9335, pp. 328–342 (2015)